PyTorch 入门笔记

基于 CS224N 的学习笔记

Keywords: Tensor / Autograd / Training Loop / nn.Module / Dropout

1 先记住主线：训练其实就 5 步

前向传播 (Forward) -> 损失计算 (Loss) -> 反向传播 (Backward) -> 参数更新 (Update) -> 梯度重置 (Reset)

2 Tensor：万物起点

2.1 三种常见创建方式

Tensor 创建与仿造创建

```python
import torch
# 1) 从 Python 列表直接创建 Tensor
# 形状是 [2, 3], dtype 会自动推断为 int64
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(x)

# 2) 按形状创建 Tensor
shape = (2, 3)
print(torch.ones(shape))   # 全 1
print(torch.randn(shape))  # 标准正态随机数

# 3) 参考已有 Tensor 创建“同形状”随机 Tensor
template = torch.tensor([[1, 2], [3, 4]])
rand_like = torch.randn_like(template, dtype=torch.float32)
print(rand_like)
```

运行结果

```
tensor([[1, 2, 3],
        [4, 5, 6]])
tensor([[1., 1., 1.],
        [1., 1., 1.]])
tensor([[ 1.3005, -0.4467,  1.4764],
        [-0.1194, -0.8527,  0.3998]])
tensor([[-0.6153,  1.0806],
```

```
        [-0.1369,  1.3699]])
```

2.2 Debug 三件套：shape / dtype / device

定位 90% 问题的最短代码

```python
t = torch.randn(2, 3)

# shape: 张量维度
print(t.shape)
# dtype: 数据类型（是否能参与梯度计算）
print(t.dtype)
# device: 在 CPU 还是 GPU
print(t.device)
```

运行结果

```
torch.Size([2, 3])
torch.float32
cpu
```

2.3 dtype 规则（新手必看）

参数和标签的 dtype 区分

```python
# 可学习参数通常是 float32，并可求导
w = torch.randn(3, 1, dtype=torch.float32, requires_grad=True)

# 分类标签通常是 int64，不需要梯度
labels = torch.tensor([0, 1, 1, 0], dtype=torch.int64)

print(w.dtype, labels.dtype)
```

运行结果

```
torch.float32
torch.int64
```

2.4 Tensor 和 NumPy 的互转与同名操作

NumPy vs PyTorch 对照

```python
import torch
import numpy as np

x_numpy = np.array([0.1, 0.2, 0.3])
x_torch = torch.tensor([0.1, 0.2, 0.3])

print(f"NumPy: {x_numpy}")  # NumPy: [0.1 0.2 0.3]
print(f"PyTorch: {x_torch}")  # PyTorch: tensor([0.1000, 0.2000, 0.3000])

# NumPy 和 PyTorch 互转
print(f"NumPy -> PyTorch: {torch.from_numpy(x_numpy)}")
# NumPy -> PyTorch: tensor([0.1000, 0.2000, 0.3000], dtype=torch.float64)

print(f"PyTorch -> NumPy: {x_torch.numpy()}")
# PyTorch -> NumPy: [0.1 0.2 0.3]

y_numpy = np.array([3, 4, 5.])
y_torch = torch.tensor([3, 4, 5.])
# x+y
print(f"NumPy: {x_numpy + y_numpy}")  # NumPy: [3.1 4.2 5.3]
print(f"PyTorch: {x_torch + y_torch}")  # PyTorch: tensor([3.1000, 4.2000,
    5.3000])

# 范数 norm
print(f"NumPy: {np.linalg.norm(x_numpy):.6g}")  # NumPy: 0.374166
print(f"PyTorch: {torch.norm(x_torch):.6g}")  # PyTorch: 0.374166

x_numpy = np.array([[1, 2], [3, 4.]])
x_torch = torch.tensor([[1, 2], [3, 4.]])
# 沿第 0 维求均值
print(f"NumPy: {np.mean(x_numpy, axis=0)}")  # NumPy: [2. 3.]
print(f"PyTorch: {torch.mean(x_torch, dim=0)}")  # PyTorch: tensor([2.,
   3.])
```

2.5 Tensor.view() 与 reshape

批量 reshape 的常见写法

```
N, C, W, H = 10000, 3, 28, 28
X = torch.randn((N, C, W, H))

print(X.shape)
# view 的参数是目标形状的各个维度，必须满足总元素数相同
print(X.view(N, C, 784).shape)  # view(10000, 3, 784)

# -1 表示自动推断该维度大小，根据总元素数推算
print(X.view(-1, C, 784).shape)
# view(-1, 3, 784) 自动推断第一个维度为10000

# reshape 的参数是目标形状，可以是元组或逗号分隔的整数
print(torch.reshape(X, (-1, C, 784)).shape)  # reshape(X, (-1, 3, 784))
```

运行结果

```
torch.Size([10000, 3, 28, 28])
torch.Size([10000, 3, 784])
torch.Size([10000, 3, 784])
torch.Size([10000, 3, 784])
```

3 Autograd：自动求导引擎

3.1 requires_grad 与反向传播

一眼看懂计算图和梯度

```python
a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(3.0, requires_grad=True)
x = torch.tensor(4.0, requires_grad=True)

# 前向: z = x * (a + b)
y = a + b  # y = 2 + 3 = 5
z = x * y  # z = 4 * 5 = 20
print(z, z.grad_fn)  # 打印张量值和计算图节点

# 反向: 从 z 开始反向计算
# Step 1: dz/dz = 1

# Step 2: dz/dy = z / y = x = 4
#         dz/dx = z / x = y = (a + b) = 5

# Step 3: dz/da = dz/dy * dy/da = 4 * 1 = 4
#         dz/db = dz/dy * dy/db = 4 * 1 = 4
z.backward()
print(a.grad, b.grad, x.grad)
```

运行结果

```
tensor(20., grad_fn=<MulBackward0>)
tensor(4.) tensor(4.) tensor(5.)
```

3.2 标量函数求导（平方函数）

一维导数验证

```python
def f(x):
    return (x - 2) ** 2

def fp(x):
    return 2 * (x - 2)

x = torch.tensor([1.0], requires_grad=True)
y = f(x)
```

```python
y.backward()

print("Analytical f'(x):", fp(x))
print("PyTorch's f'(x):", x.grad)
```

运行结果

```
Analytical f'(x): tensor([-2.], grad_fn=<MulBackward0>)
PyTorch's f'(x): tensor([-2.])
```

4 常见易错点

4.1 第一：* 和 @ 不是一回事

逐元素乘法 vs 矩阵乘法

```python
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[10, 20], [30, 40]])

# * 是逐元素乘法
print(a * b)

m1 = torch.tensor([[1, 2, 3], [4, 5, 6]])
m2 = torch.tensor([[7, 8], [9, 10], [11, 12]])

# @ 是矩阵乘法
print(m1 @ m2)
```

运行结果

```
tensor([[ 10,  40],
        [ 90, 160]])
tensor([[ 58,  64],
        [139, 154]])
```

4.2 第二：dim 一定要想清楚

dim=0 与 dim=1

```python
scores = torch.tensor([[10., 20., 30.], [5., 10., 15.]])

# dim=0: 沿着行压缩 -> 每列均值
print(scores.mean(dim=0))
# dim=1: 沿着列压缩 -> 每行均值
print(scores.mean(dim=1))
```

运行结果

```
tensor([ 7.5000, 15.0000, 22.5000])
tensor([20., 10.])
```

4.3 第三：索引和切片要敢打印 shape

基础切片 + 高级索引

```python
x = torch.tensor([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]],
    [[9, 10], [11, 12]]
], dtype=torch.float32)

# 选出每个样本的 [0,0] 位置
print(x[:, 0, 0])

# 高级索引：重复选取第 0 和第 1 个样本
i = torch.tensor([0, 0, 1, 1])
print(x[i])
```

运行结果

```
tensor([1., 5., 9.])
tensor([[[1., 2.],
         [3., 4.]],

        [[1., 2.],
         [3., 4.]],

        [[5., 6.],
         [7., 8.]],

        [[5., 6.],
         [7., 8.]]])
```

5 手写训练循环：线性回归走一遍

5.1 先造数据，再定义可学习参数

准备训练样本

```python
N, D_in, D_out = 10, 1, 1

# 造输入
X = torch.randn(N, D_in)

# 造真实线性关系 y = 2x + 1 + 噪声
w_true = torch.tensor([[2.]])
b_true = torch.tensor([[1.]])
y_true = X * w_true + b_true + torch.randn(N, D_out) * 0.1

# 初始化待学习参数
w = torch.randn(D_in, D_out, requires_grad=True)
b = torch.randn(1, requires_grad=True)

print(X.shape, y_true.shape)
```

运行结果

```
torch.Size([10, 1]) torch.Size([10, 1])
```

5.2 完整 5 步训练循环

手动版训练循环

```python
lr = 0.01
epochs = 100

for epoch in range(epochs):
    # 1) Forward
    y_hat = X @ w + b
    # 2) Loss
    loss = ((y_hat - y_true) ** 2).mean()
    # 3) Backward
    loss.backward()

    # 4) Update（注意 no_grad）
    with torch.no_grad():
        w -= lr * w.grad
        b -= lr * b.grad

    # 5) Reset
    w.grad.zero_()
    b.grad.zero_()

    if epoch % 20 == 0:
        print(f"epoch={epoch:03d}, loss={loss.item():.4f}")
```

运行结果

```
epoch=00, loss=3.4021
epoch=20, loss=0.5038
epoch=40, loss=0.0867
epoch=60, loss=0.0201
epoch=80, loss=0.0091
```

重点：参数更新必须放在 `torch.no_grad()` 里，否则更新动作会进入计算图，内存占用会持续增长。

6 工程化升级：nn.Module

6.1 定义模型类：线性回归

用 nn.Module 封装线性回归

```python
import torch.nn as nn

# 继承 nn.Module，所有模型都需要继承它
class LinearRegressionModel(nn.Module):
    # 初始化：定义模型结构
    def __init__(self, in_features, out_features):
        # 必须调用父类初始化
        super().__init__()
        # 把参数管理交给 nn.Linear（自动处理 weight 和 bias）
        self.linear_layer = nn.Linear(in_features, out_features)

    # 前向传播：定义数据流向
    def forward(self, x):
        return self.linear_layer(x)

# 创建模型实例：输入维度 1，输出维度 1
model = LinearRegressionModel(1, 1)
print(model)
```

运行结果

```
LinearRegressionModel(
  (linear_layer): Linear(in_features=1, out_features=1, bias=True)
)
```

前后对比：

- **前面（手动版）**：手动定义 w 和 b 参数，手动写 5 步循环。
- **现在（nn.Module 版）**：参数由 nn.Linear 自动管理，模型封装在类中，前向传播只需调用 forward() 方法
- **后续优化**：结合 optim 优化器，参数更新和梯度清零都自动化，训练循环大幅简化

6.2 定义模型类：多层感知机

用 nn.Module + nn.Sequential 搭建 MLP

```python
import torch
import torch.nn as nn

class MultilayerPerceptron(nn.Module):

    def __init__(self, input_size, hidden_size):
        # 调用父类 nn.Module 的初始化方法
        super(MultilayerPerceptron, self).__init__()

        # 保存初始化参数
        self.input_size = input_size
        self.hidden_size = hidden_size

        # 定义模型结构
        # self.model 的命名没有特殊要求，可以是任意名称
        self.model = nn.Sequential(
            nn.Linear(self.input_size, self.hidden_size),
            nn.ReLU(),
            nn.Linear(self.hidden_size, self.input_size),
            nn.Sigmoid()
        )

    def forward(self, x):
        # 前向传播：将输入通过模型得到输出
        output = self.model(x)
        return output


mlp_model = MultilayerPerceptron(input_size=10, hidden_size=5)
print(mlp_model)

# 使用样本输入进行测试
test_input = torch.randn(1, 10)  # 批次大小为 1，输入特征数为 10
test_output = mlp_model(test_input)
print(f"\\n测试输入形状：{test_input.shape}")
print(f"测试输出形状：{test_output.shape}")
print(f"测试输出：{test_output}")
```

运行结果

```
MultilayerPerceptron(
  (model): Sequential(
    (0): Linear(in_features=10, out_features=5, bias=True)  % 线性层: 10
       -> 5
    (1): ReLU()  % 激活函数: 负数截断
    (2): Linear(in_features=5, out_features=10, bias=True)  % 线性层: 5 ->
        10
    (3): Sigmoid()  % 激活函数: 输出概率
  )
)

测试输入形状: torch.Size([1, 10])
测试输出形状: torch.Size([1, 10])
测试输出: tensor([[...]], grad_fn=<SigmoidBackward0>)
```

7 优化器 + 损失函数

7.1 MSELoss 最小例子

均方误差

```
mse_loss_fn = nn.MSELoss()
input = torch.tensor([[0., 0, 0]])
target = torch.tensor([[1., 0, -1]])

# 计算预测值与目标值之间的损失
# MSELoss计算方式: ((0-1)^2+(0-0)^2+(0-(-1))^2)/3 = (1 + 0 + 1)/3 = 2/3
loss = mse_loss_fn(input, target)
print(loss)
```

运行结果

```
tensor(0.6667)
```

7.2 CrossEntropyLoss 输入输出格式

三条样本的分类损失

```
loss = nn.CrossEntropyLoss()

# 输入: 3 个样本, 每个样本 2 个类别的未归一化分数 (logits)
input = torch.tensor([[-1., 1], [-1, 1], [1, -1]])

# 目标: 3 个样本的真实类别标签 (0 或 1)
target = torch.tensor([1, 1, 0])
# CrossEntropyLoss 会对 logits 做 softmax, 然后计算交叉熵

# 样本 1: logits = [-1, 1]
# softmax 计算步骤:
#   1) 指数化: e^(-1)   0.3679, e^1   2.7183
#   2) 求和: 0.3679 + 2.7183   3.0862
#   3) 归一化: [0.3679/3.0862, 2.7183/3.0862]   [0.1192, 0.8808]
# softmax = [0.1192, 0.8808]
# target = 1 → 取第二个概率 0.8808
# loss_1 = -log(0.8808)   0.1269

# 样本 2: logits = [-1, 1]
# 计算步骤: 同上
```

```python

# 样本 3: logits = [1, -1]
# softmax 计算步骤:
#   1) 指数化: e^1   2.7183, e^(-1)   0.3679
#   2) 求和: 2.7183 + 0.3679   3.0862
#   3) 归一化: [2.7183/3.0862, 0.3679/3.0862]   [0.8808, 0.1192]
# softmax = [0.8808, 0.1192]
# target = 0 → 取第一个概率 0.8808
# loss_3 = -log(0.8808)   0.1269

# batch loss = (0.1269 + 0.1269 + 0.1269) / 3   0.1269
output = loss(input, target)
print(output)
```

运行结果

```
tensor(0.1269)
```

7.3 优化器部分

从手写更新到 Adam

```python
import torch.optim as optim

# 这里不再手写 W -= learning_rate * W.grad。
# 改用 torch.optim 提供的优化器来封装更新逻辑，并传入模型参数与学习率。
# optim.Adam 会为不同参数自适应调整学习率。
optimizer = optim.Adam(model.parameters(), lr=0.01)

# 同时使用 torch.nn 提供的现成损失函数
loss_fn = nn.MSELoss()  # 均方误差损失
```

7.4 专业训练循环：三行真言

最常见的训练模板

```python
for epoch in range(100):
    # 前向传播 + 计算损失
    y_hat = model(X)
    loss = loss_fn(y_hat, y_true)

    # 三行真言
    optimizer.zero_grad()  # 1) 清空上一轮累积的梯度
    loss.backward()        # 2) 计算当前轮次梯度
    optimizer.step()       # 3) 更新参数

    if epoch % 10 == 0:
        w_learned = model.linear_layer.weight.item()
        b_learned = model.linear_layer.bias.item()
        print(f"轮次 {epoch:02d}: 损失={loss.item():.4f}, W={w_learned:.3f}, b={b_learned:.3f}")
```

运行结果

```
轮次 00: 损失=0.0069, W=1.998, b=1.007
轮次 10: 损失=0.0069, W=1.998, b=1.007
轮次 20: 损失=0.0069, W=1.998, b=1.007
轮次 30: 损失=0.0069, W=1.998, b=1.007
轮次 40: 损失=0.0069, W=1.998, b=1.007
轮次 50: 损失=0.0069, W=1.998, b=1.007
轮次 60: 损失=0.0069, W=1.998, b=1.007
轮次 70: 损失=0.0069, W=1.998, b=1.007
轮次 80: 损失=0.0069, W=1.998, b=1.007
轮次 90: 损失=0.0069, W=1.998, b=1.007
```

8 从基础到工程：完整训练流程示例

本节按照”采样策略 → 模型表达能力 → 优化器动力学”的顺序组织。核心目的是：训练循环模板不变（前向、反向、更新三步不变），但当我们改变数据使用方式、模型结构或优化器超参数时，收敛曲线与最终效果会出现可解释的差异。

8.1 用 nn.Linear 做全量 GD

本小节在做什么： 先建立一个最小基线：在线性可拟合的数据上，使用 nn.Linear 和全量梯度下降（每一步用全部样本）训练。这样写的目的，是先给出一个”梯度稳定、收敛平滑”的参照系，方便后续和 SGD 及非线性任务对比。

看到的现象： loss 下降较平滑，参数逐步逼近真值 true_w。这是因为全量 GD 的梯度来自全数据平均，方差较小，更新方向更稳定。

PyTorch GD 版线性回归

```python
import numpy as np
import torch
import torch.nn as nn

d = 2
step_size = 0.1

X = torch.randn(50, d)
true_w = torch.tensor([[-1.0], [2.0]])
y = X @ true_w + torch.randn(50, 1) * 0.1

linear_module = nn.Linear(d, 1, bias=True)
loss_func = nn.MSELoss()
optim = torch.optim.SGD(linear_module.parameters(), lr=step_size)

print("iter,\\tloss,\\tw")
for i in range(20):
    y_hat = linear_module(X)
    loss = loss_func(y_hat, y)
    optim.zero_grad()
    loss.backward()
    optim.step()
    print("{},\\t{:.2f},\\t{}".format(
        i, loss.item(), linear_module.weight.view(2).detach().numpy()))

print("\\ntrue w\\t\\t", true_w.view(2).numpy())
print("estimated w\\t", linear_module.weight.view(2).detach().numpy())
```

运行结果

```
iter,  loss,  w
0,  5.93,  [0.450549  0.2314362]
1,  4.14,  [0.2404964 0.5295114]
2,  2.92,  [0.06231812 0.7730595 ]
3,  2.09,  [-0.08906882  0.97288895]
4,  1.51,  [-0.21788383  1.1375176 ]
5,  1.09,  [-0.32764053  1.2736821 ]
6,  0.80,  [-0.42127335  1.3867309 ]
7,  0.59,  [-0.5012397  1.4809269]
8,  0.44,  [-0.569603   1.5596819]
9,  0.33,  [-0.6280999  1.6257378]
10,  0.25,  [-0.6781955  1.6813077]
11,  0.19,  [-0.721128   1.7281855]
12,  0.14,  [-0.75794625  1.7678317 ]
13,  0.11,  [-0.78954  1.80144]
14,  0.08,  [-0.8166653  1.8299907]
15,  0.07,  [-0.8399655  1.8542919]
16,  0.05,  [-0.85998887  1.8750123 ]
17,  0.04,  [-0.87720305  1.8927073 ]
18,  0.04,  [-0.89200747  1.9078401 ]
19,  0.03,  [-0.9047436  1.9207981]

true w  [-1.  2.]
estimated w  [-0.9047436  1.9207981]
```

8.2 随机采样点做 SGD

与上一小节的关键不同点： 这一节主要改变”采样策略”：每一步只随机取一个样本更新参数（SGD），而不是用全量样本。这样写是为了直观看到计算成本与梯度噪声之间的权衡：单步更便宜，但曲线通常更抖动。

为什么会抖动但仍能收敛： 单样本梯度是全梯度的带噪估计，方向会有随机波动，所以 loss 常出现上下起伏；但在合适学习率下，长期期望方向仍指向较优解，因此总体趋势仍会下降。

PyTorch SGD 版线性回归

```
step_size = 0.01
linear_module = nn.Linear(d, 1)
loss_func = nn.MSELoss()
optim = torch.optim.SGD(linear_module.parameters(), lr=step_size)
print("iter,\\tloss,\\tw")
for i in range(200):
    rand_idx = np.random.choice(50)
    x = X[rand_idx]
    y_hat = linear_module(x)
    loss = loss_func(y_hat, y[rand_idx])
    optim.zero_grad()
    loss.backward()
    optim.step()
    if i % 20 == 0:
        print("{},\\t{:.2f},\\t{}".format(
            i, loss.item(), linear_module.weight.view(2).detach().numpy())
)
print("\\ntrue w\\t\\t", true_w.view(2).numpy())
print("estimated w\\t", linear_module.weight.view(2).detach().numpy())
```

运行结果

```
iter, loss, w
0, 2.73, [ 0.24269418 -0.6256218 ]
20, 0.47, [0.01831682 0.11636729]
40, 2.37, [-0.21397021  0.85143703]
60, 0.03, [-0.33021447  1.2125181 ]
80, 1.04, [-0.45070502  1.393761  ]
100, 0.14, [-0.65619284  1.5554723 ]
120, 0.02, [-0.73517257  1.6326638 ]
140, 0.29, [-0.79365164  1.7716284 ]
160, 0.03, [-0.87195843  1.8720678 ] # true w [-1.  2.]
180, 0.01, [-0.91041076  1.9069777 ] # estimated w [-0.9285  1.9488]
```

9 非线性与优化器

9.1 两层隐藏层网络拟合非线性函数

为什么要从线性模型切到两层网络： 前两节说明了”同一线性任务下，不同采样策略会影响优化轨迹”；这一节进一步说明另一条主线：当目标函数本身是非线性的，模型表达能力会成为首要瓶颈。因此改用两层 Tanh 网络，让模型具备逼近复杂非线性关系的能力。
本节想传达的要点： 即使采用相同的训练流程（zero_grad/backward/step），只要模型的参数容量（隐层宽度）足够大，神经网络的表达能力就足以逼近非线性函数，loss 损失值也会持续下降；
这说明模型训练的成功与否，既取决于优化算法本身，也取决于模型架构的表达能力，两者是互补的，缺一不可。

拟合非线性目标函数

```
d = 1
n = 200
X = torch.rand(n, d)
y = 4 * torch.sin(np.pi * X) * torch.cos(6 * np.pi * X ** 2)

step_size = 0.05
n_epochs = 6000
n_hidden_1 = 32
n_hidden_2 = 32
d_out = 1

neural_network = nn.Sequential(
    nn.Linear(d, n_hidden_1),
    nn.Tanh(),
    nn.Linear(n_hidden_1, n_hidden_2),
    nn.Tanh(),
    nn.Linear(n_hidden_2, d_out)
)

loss_func = nn.MSELoss()
optim = torch.optim.SGD(neural_network.parameters(), lr=step_size)
```

训练

```python
print("iter,\\tloss")
for i in range(n_epochs):
    y_hat = neural_network(X)
    loss = loss_func(y_hat, y)
    optim.zero_grad()
    loss.backward()
    optim.step()
    if i % (n_epochs // 10) == 0:
        print("{},\\t{:.2f}".format(i, loss.item()))
```

运行结果

```
iter,loss
000,4.30
600,3.24
1200,1.36
1800,0.84
2400,0.47
3000,0.26
3600,0.21
4200,0.17
4800,0.13
5400,0.11
```

9.2 SGD + Momentum

与上一小节的关键不同点

网络结构保持同类两层 MLP，不再改变任务定义，主要改优化器：从纯 SGD 增加 momentum=0.9。这样写的目的，是把焦点放在”优化动力学”上：动量会累积历史更新方向，减小来回震荡，并在一致方向上加速前进。

结果应如何解读

从输出可见，使用动量后在更少迭代内达到更低 loss，体现了更快的有效收敛。这说明在工程实践中，优化器超参数往往是提升训练效率的低成本杠杆。

公平性说明

本节将 n_epochs 从 6000 改为 1500，意图是演示”更快收敛”的现象，而非做严格 benchmark。若做严格对比，应固定随机种子、初始化、训练轮数与日志间隔，再比较有无动量的曲线。

只改优化器超参数

```
step_size = 0.05
momentum = 0.9
n_epochs = 1500

neural_network = nn.Sequential(
    nn.Linear(1, 32),
    nn.Tanh(),
    nn.Linear(32, 32),
    nn.Tanh(),
    nn.Linear(32, 1)
)
loss_func = nn.MSELoss()
optim = torch.optim.SGD(neural_network.parameters(), lr=step_size,
    momentum=momentum)

print("iter,\\tloss")
for i in range(n_epochs):
    y_hat = neural_network(X)
    loss = loss_func(y_hat, y)
    optim.zero_grad()
    loss.backward()
    optim.step()
    if i % (n_epochs // 10) == 0:
        print("{},\\t{:.2f}".format(i, loss.item()))
```

运行结果

```
iter,loss
0,4.31
150,1.17
300,0.33
450,0.12
600,0.07
750,0.05
900,0.04
1050,0.04
1200,0.03
1350,0.01
```

本节小结

- **采样策略（GD vs SGD）**：决定单步计算成本与梯度噪声大小。
- **模型容量（线性层 vs 多层网络）**：决定任务是否“可表示、可拟合”。
- **优化器动力学（SGD vs Momentum）**：决定在可拟合前提下，收敛速度与稳定性。

因此，工程训练中常见调参顺序是：先确认任务与模型匹配，再选合适优化器，最后在计算预算内平衡批大小与学习率。

10 常用神经网络模块

10.1 Linear 层最小例子

线性层参数与输出

```python
import torch

# 定义输入和输出维度
D_in = 1
D_out = 1
# 创建线性层：将 D_in 维输入映射到 D_out 维输出
linear_layer = torch.nn.Linear(D_in, D_out)

# 生成示例输入数据：10 个样本，每个样本 D_in 维
X = torch.randn(10, D_in)

print(f"w:{linear_layer.weight}")
print(f"b:{linear_layer.bias}")

# 通过线性层进行前向传播
y_hat_nn = linear_layer(X)
print("y_hat 形状:", y_hat_nn.shape)

# Linear 层的用途：
# 1. 定义线性变换：y = X @ W^T + b，其中 W 和 b 是可学习参数
# 2. 自动管理参数：nn.Linear内部自动创建权重矩阵W和偏置向量b，无需手动初始化
# 3. 前向传播：调用 linear_layer(X) 直接计算线性变换，等价于X@ W^T+b
# 4. 梯度计算：参数自动纳入计算图，反向传播时自动计算梯度
# 5. 模型组件：作为神经网络的基础层，可以堆叠多层 Linear 构建深层网络
```

运行结果

```
w:Parameter containing:
tensor([[0.0560]], requires_grad=True)
b:Parameter containing:
tensor([0.9768], requires_grad=True)
y_hat 形状: torch.Size([10, 1])
```

10.2 ReLU 激活函数

负数截断示例

```python
import torch
relu = torch.nn.ReLU()
sample_data = torch.tensor([-2.0, -0.5, 0.0, 0.5, 2.0])
# 通过 ReLU 函数进行激活（负数变为 0，正数保持不变）
activated_data = relu(sample_data)
print(activated_data)
```

运行结果

```
tensor([0.0000, 0.0000, 0.0000, 0.5000, 2.0000])
```

10.3 Softmax 激活函数

dim=-1 的概率归一化

```python
# 其中 dim=-1 表示在最后一个维度上进行操作
# 对于 softmax，dim=-1 意味着在最后一个维度上进行 softmax 归一化
softmax = torch.nn.Softmax(dim=-1)

sample_probs = torch.tensor([[1.0, 2.0, 3.0], [1.0, 1.0, 1.0]])
result = softmax(sample_probs)
print(result)
```

运行结果

```
tensor([[0.0900, 0.2447, 0.6652],
        [0.3333, 0.3333, 0.3333]])
```

10.4 Sigmoid 激活函数

sigmoid 的概率映射

```python
import torch
sigmoid = torch.nn.Sigmoid()
sample_data = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
# sigmoid 函数: 1 / (1 + e^(-x))
# 将样本数据通过 sigmoid 函数进行映射, 输出值在 (0, 1) 之间
# 执行前向传播, 得到经过 sigmoid 激活的结果
result = sigmoid(sample_data)
print(result)
```

运行结果

```
tensor([0.1192, 0.2689, 0.5000, 0.7311, 0.8808])
```

10.5 Embedding 层

词 ID 到向量

```python
# 定义词表大小和嵌入维度
vocab_size = 10
embedding_dim = 3
# 创建 Embedding 层, 将词 ID 映射到向量
embedding_layer = torch.nn.Embedding(vocab_size, embedding_dim)
# 输入词 ID 序列
input_ids = torch.tensor([[1, 5, 0, 8]])
# 将词 ID 转换为对应的嵌入向量
word_vectors = embedding_layer(input_ids)
# 输出向量形状: [batch_size, seq_length, embedding_dim]
print(word_vectors.shape)
```

运行结果

```
torch.Size([1, 4, 3])
```

10.6 LayerNorm 层

按最后一维归一化

```python
# LayerNorm 在最后一个维度（dim=-1）上进行归一化
norm_layer = torch.nn.LayerNorm(normalized_shape=3)
input_features = torch.tensor([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])
normalize_features = norm_layer(input_features)

print(normalize_features)
print(normalize_features.mean(dim=-1))
print(normalize_features.std(dim=-1, unbiased=False))
```

运行结果

```
tensor([[-1.2247,  0.0000,  1.2247],
    [-1.2247,  0.0000,  1.2247]], grad_fn=<NativeLayerNormBackward0>)
tensor([0., 0.], grad_fn=<MeanBackward1>)
tensor([1.0000, 1.0000], grad_fn=<StdBackward0>)
```

10.7 Dropout 层

训练/评估模式下的差异

```python
dropout_layer = torch.nn.Dropout(p=0.5)
input_tensor = torch.ones(1, 10)

dropout_layer.train()
output_during_train = dropout_layer(input_tensor)
print("训练模式输出: ")
print(output_during_train)

dropout_layer.eval()
output_during_eval = dropout_layer(input_tensor)
print("评估模式输出: ")
print(output_during_eval)
```

运行结果

```
训练模式输出:
tensor([[0., 2., 0., 2., 2., 0., 2., 0., 0., 2.]])
评估模式输出:
tensor([[1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]])
```

11 训练模式 vs 推理模式（实战必会）

这部分是教程里非常重要但容易在入门笔记里遗漏的实践点：

- `model.train()`：训练模式（Dropout 开启，BatchNorm 使用 batch 统计量）
- `model.eval()`：推理模式（Dropout 关闭，BatchNorm 使用滑动统计量）
- `with torch.no_grad():`：推理时关闭梯度图，省显存、提速
- `tensor.item()`：把 0-dim tensor 转 Python 标量
- `tensor.detach().cpu().numpy()`：把张量安全转换成 NumPy

Dropout 在 train/eval 下行为不同

```python
import numpy as np
import torch
import torch.nn as nn

# 固定随机种子，便于复现对比
torch.manual_seed(7)
demo = nn.Sequential(
    nn.Linear(4, 4),
    nn.ReLU(),
    nn.Dropout(p=0.5),
    nn.Linear(4, 2),
)
x_demo = torch.ones(3, 4)

# 训练模式：Dropout 生效
demo.train()
out_train_1 = demo(x_demo)
out_train_2 = demo(x_demo)
print("训练模式下两次前向是否一致？", torch.allclose(out_train_1,
    out_train_2))

# 推理模式：Dropout 关闭，输出稳定
demo.eval()
out_eval_1 = demo(x_demo)
out_eval_2 = demo(x_demo)
print("推理模式下两次前向是否一致？", torch.allclose(out_eval_1,
    out_eval_2))

# item(): 标量提取
scalar_loss = torch.tensor(3.14)
```

```python
print("标量张量: ", scalar_loss)
print("通过 item() 取出的 Python 标量: ", scalar_loss.item())

# detach + cpu + numpy: 安全转 numpy
t = torch.randn(2, 3, requires_grad=True)
np_array = t.detach().cpu().numpy()
print("numpy 形状: ", np_array.shape)
print("numpy 数据类型: ", np_array.dtype)
```

运行结果

```
训练模式下两次前向是否一致? False
推理模式下两次前向是否一致? True
标量张量: tensor(3.1400)
通过 item() 取出的 Python 标量: 3.140000104904175
numpy 形状: (2, 3)
numpy 数据类型: float32
```

推理阶段建议固定写法：`model.eval()` + `with torch.no_grad():`。这会更稳、更省显存。

12 补充：原文未覆盖内容（放最后）

12.1 Conv2d 维度变化

输入输出形状

```
im_channels = 3 # 若使用 RGB 图像，输入通道数为 3；黑白图像则为 1
out_channels = 16 # 这是可调超参数
kernel_size = 3 # 这也是可调超参数
batch_size = 4
image_width = 32
image_height = 32

im = torch.randn(batch_size, im_channels, image_width, image_height)
m = nn.Conv2d(im_channels, out_channels, kernel_size)
convolved = m(im) # m 是模块对象，可像函数一样调用

print("输入图像形状", im.shape)
print("卷积后图像形状", convolved.shape)
```

运行结果

```
输入图像形状 torch.Size([4, 3, 32, 32])
卷积后图像形状 torch.Size([4, 16, 30, 30])
```

12.2 RNN 词预测小例子

Embedding + RNN + Linear

```
def process_corpus(corpus, sentence_length):
    """
    参数:
        corpus (str) -- 连续文本，可以是任意内容，但建议足够长。
        sentence_length (int) -- 输出中每个句子的长度，不足会自动填充。
    返回:
        x_idx, y_idx, word_to_idx, idx_to_word
    """
    # 先把语料拆成词列表
    corpus = corpus.split()

    # 再按 sentence_length 切分为更短的句子
    x = []
    y = []
```

```python
    for idx in range(0, len(corpus), sentence_length):
        x.append(corpus[idx: idx + sentence_length])
        # 目标是预测下一个词，因此 y 可看作 x 向后平移一位
        y.append(corpus[idx + 1: idx + sentence_length + 1])
    # 最后一句可能不足长度，需要做填充
    x[-1] += [""] for _ in range(sentence_length - len(x[-1]))]
    y[-1] += [""] for _ in range(sentence_length - len(y[-1]))]

    # 构建词到索引与索引到词的双向字典
    idx_to_word = {0: ""}
    word_to_idx = {"": 0}
    idx = 1
    for sentence in x:
        for word in sentence:
            if word not in word_to_idx:
                word_to_idx[word] = idx
                idx_to_word[idx] = word
                idx += 1

    x_idx = torch.tensor([[word_to_idx[w] for w in s] for s in x]).long()
    y_idx = torch.tensor([[word_to_idx[w] for w in s] for s in y]).long()
    return x_idx, y_idx, word_to_idx, idx_to_word

# 可自行调整这些参数
embedding_size = 10
sentence_length = 5
hidden_size = 5
n_epochs = 1000

# 数据集
corpus = "Hey, you. You're finally awake. You were trying to cross the border, right? " \
    "Walked right into that Imperial ambush, same as us, and that thief over there. " \
    "Skyrim was fine until you came along. Empire was nice and lazy."
x, y, word_to_idx, idx_to_word = process_corpus(corpus, sentence_length)

model_rnn = nn.Sequential(
    nn.Embedding(len(idx_to_word), embedding_size),
    nn.RNN(embedding_size, hidden_size, batch_first=True),
)
# 线性层单独定义，因为我们只取 RNN 的第一个返回值作为输入
linear = nn.Linear(hidden_size, len(idx_to_word))
print(model_rnn)
```

```python

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(list(model_rnn.parameters()) + list(linear.parameters()))

for i in range(n_epochs):
    x_mid, _ = model_rnn(x)
    y_hat = linear(x_mid).transpose(1, 2)
    loss = criterion(y_hat, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if i % (n_epochs // 10) == 0:
        print("{},\\t{:.2f}".format(i, loss.item()))

# 看看第一个句子的预测结果
with torch.no_grad():
    y_hat = linear(model_rnn(x)[0])
    y_hat = torch.argmax(y_hat, dim=2)
    sentences_hat = [[idx_to_word[int(w)] for w in s] for s in y_hat]
    sentences_true = [[idx_to_word[int(w)] for w in s] for s in y]

    sentence_idx = 0
    print(f"真实: {sentences_true[sentence_idx]}")
    print(f"预测: {sentences_hat[sentence_idx]}")

# 用一个随机起始词生成自定义句子。
with torch.no_grad():
    word = "horse"
    sentence = f"{word}"
    for _ in range(15):
        word_idx = torch.tensor([word_to_idx[word]]).reshape(1, 1)
        y_hat, _ = model_rnn(word_idx)
        y_hat = linear(y_hat)
        y_hat = torch.argmax(y_hat, dim=2)
        word = idx_to_word[y_hat.reshape(1).item()]
        sentence += f" {word}"
    print(sentence)
```

运行结果

```
Sequential(
  (0): Embedding(60, 10)
  (1): RNN(10, 5, batch_first=True)
```

```
)
000, 4.25
100, 3.79
200, 3.33
300, 2.92
400, 2.58
500, 2.28
600, 2.05
700, 1.86
800, 1.71
900, 1.59
真实: ['you.', "You're", 'finally', 'awake.', 'You']
预测: ['there.', "You're", 'finally', 'that', 'us,']
horse and that thief over there. skyrim was fine until you came along.
```